

自動化 Dark Mode 實戰：從 7 天到 24 小時

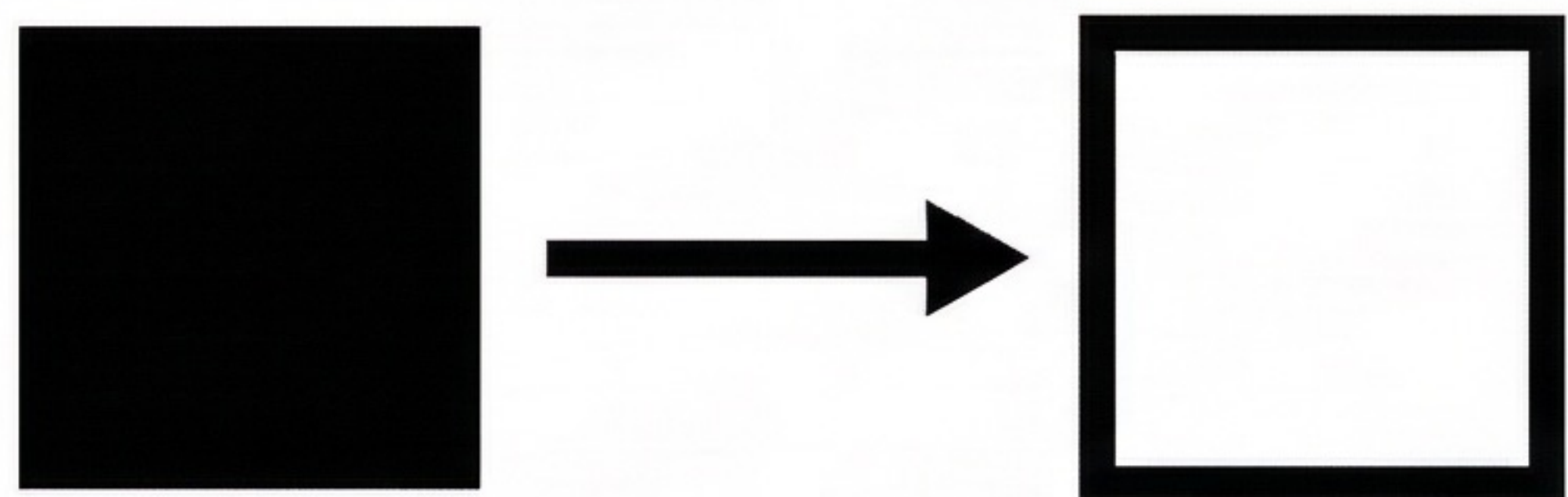
當 Figma MCP 遇見 Chrome DevTools MCP，Agent 如何接管繁瑣的前端開發？

[7 DAYS] >>> [24 HOURS]

TARGET: Mediatek Copilot

歷史包袱：為什麼不能一鍵完美切換？

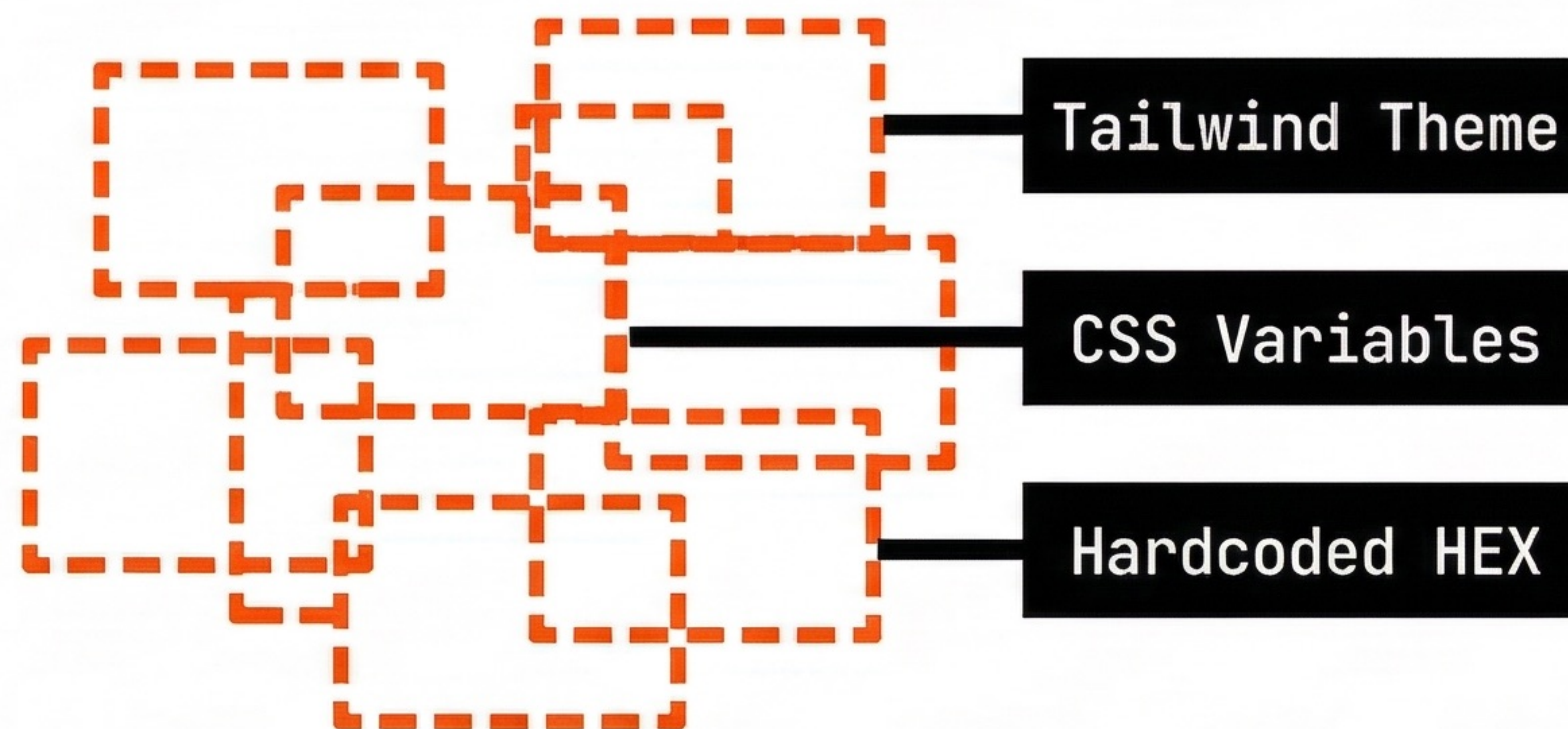
[EXPECTATION]



`filter: invert(1);`

以為只是簡單的黑白互換。

[REALITY - ERROR]



設計師具備精細的色階定義，Light/Dark 之間並非 1:1 映射。缺乏 Single Source of Truth。



[WARNING] 溝通成本無限循環：肉眼對照 -> 修改代碼 -> 重新編譯 -> 人工檢查

典範轉移：Agent 的感知與行動框架



> STATUS: Agent 不再只是寫 Code，而是具備「主動觀察」與「驗證結果」的閉環能力。

Figma MCP：直連設計源頭的光學感測器

[Figma Design System]

└ [Page: Components]

└ [Frame: Button/Primary]

└ [Group: Container]

└ [Vector: Icon]

└ [Shape: Background]

└ [Vector: Icon]

└ [Group: Icon]

└ [Group: Icon]

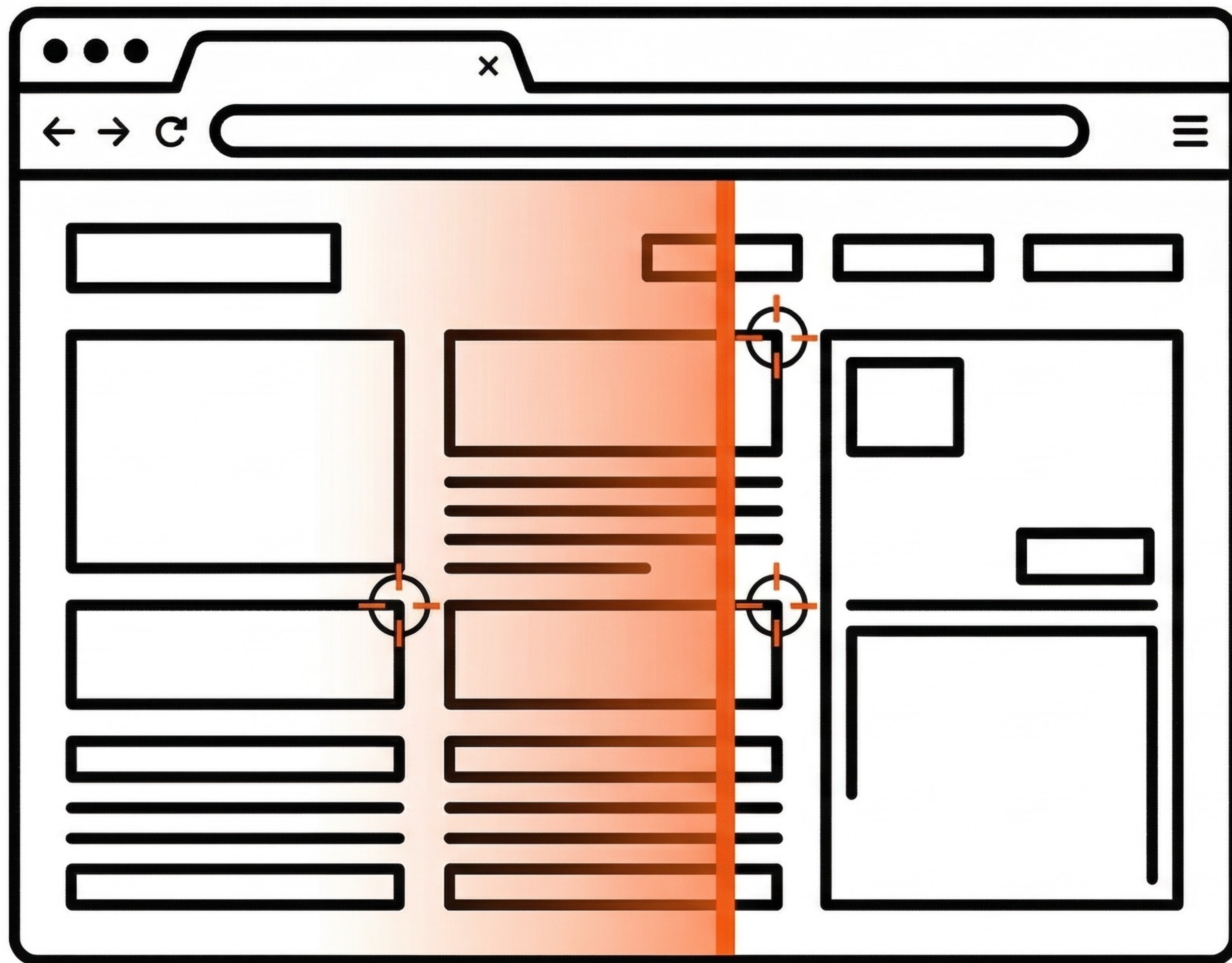
```
{
  "name": "Button/Primary",
  "type": "FRAME",
  "children": [
    {
      "name": "Container",
      "type": "GROUP",
      "children": [
        {
          "name": "Background",
          "type": "RECTANGLE",
          "styles": {
            "fill": {
              "type": "SOLID",
              "color": {
                "r": 0.98,
                "g": 0.29,
                "b": 0.00,
                "a": 1.0,
                "hex": "#FF4500"
              }
            },
            "cornerRadius": 4
          }
        },
        {
          "name": "Icon",
          "type": "VECTOR"
        }
      ]
    }
  ]
}
```

[SYS_REQ] 獲取最精確的原始 Styles 與 Tokens。

[SEC_OPT] 採用官方版 MCP：確保資料完整，無第三方阻擋風險。

[RESULT] Agent 如同設計師般解析 Layout，還原度趨近 100%。

Chrome DevTools MCP : Agent 的執行環境探測儀



Features Dashboard

即時截圖 (Visual Perception) :
得知 CSS 渲染後的真實狀態。

動態觀察 (Runtime State) :
超越靜態 Code，掌握執行時的視覺。

操控能力 (Execution) :
繞過 Sandbox 限制，自動導航與網路請求。

[UPDATE PATCH] Chrome 144+ 支援
直連當前視窗，環境配置零門檻。

核心指令：定義 dark-mode-sync 工作流

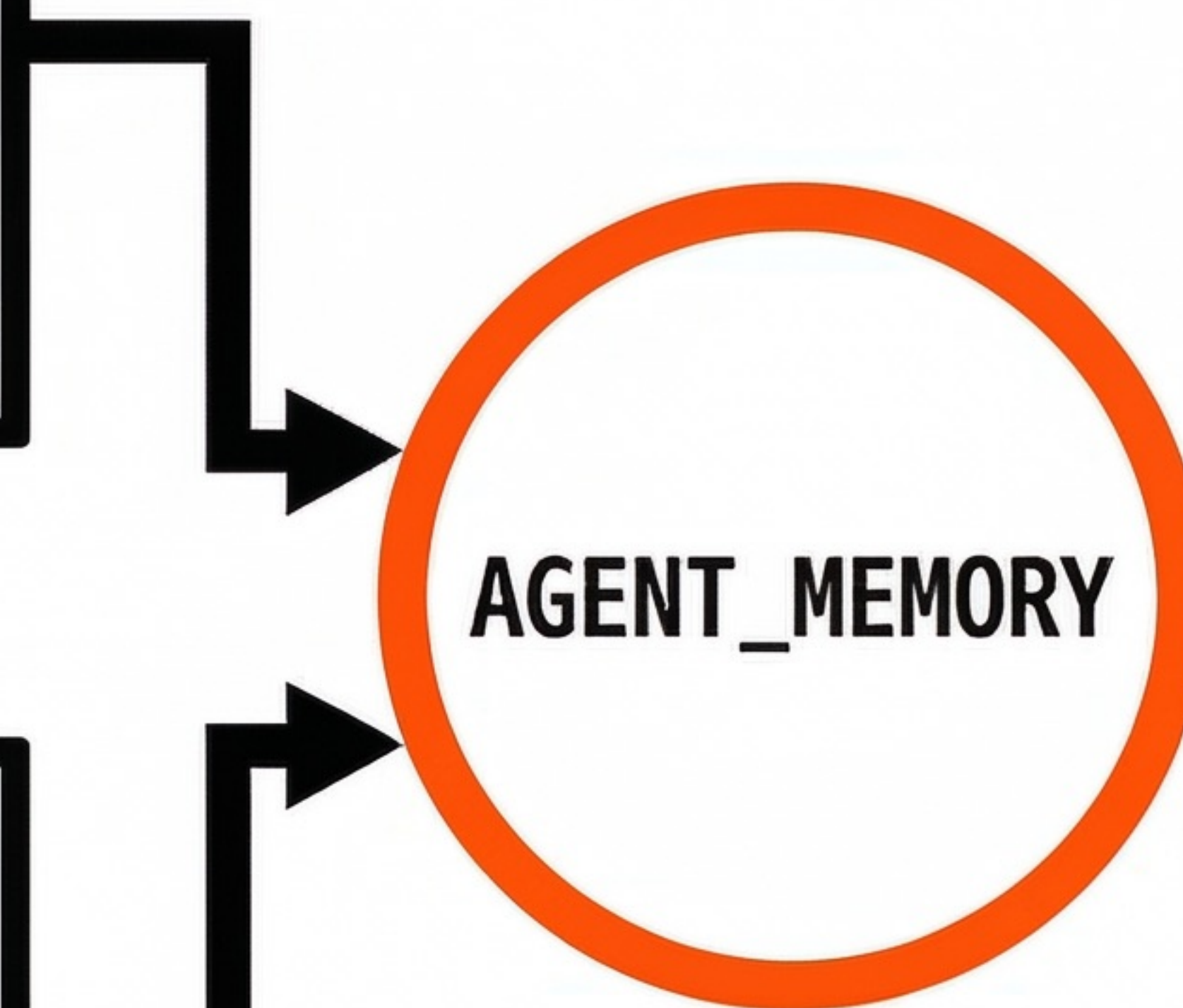
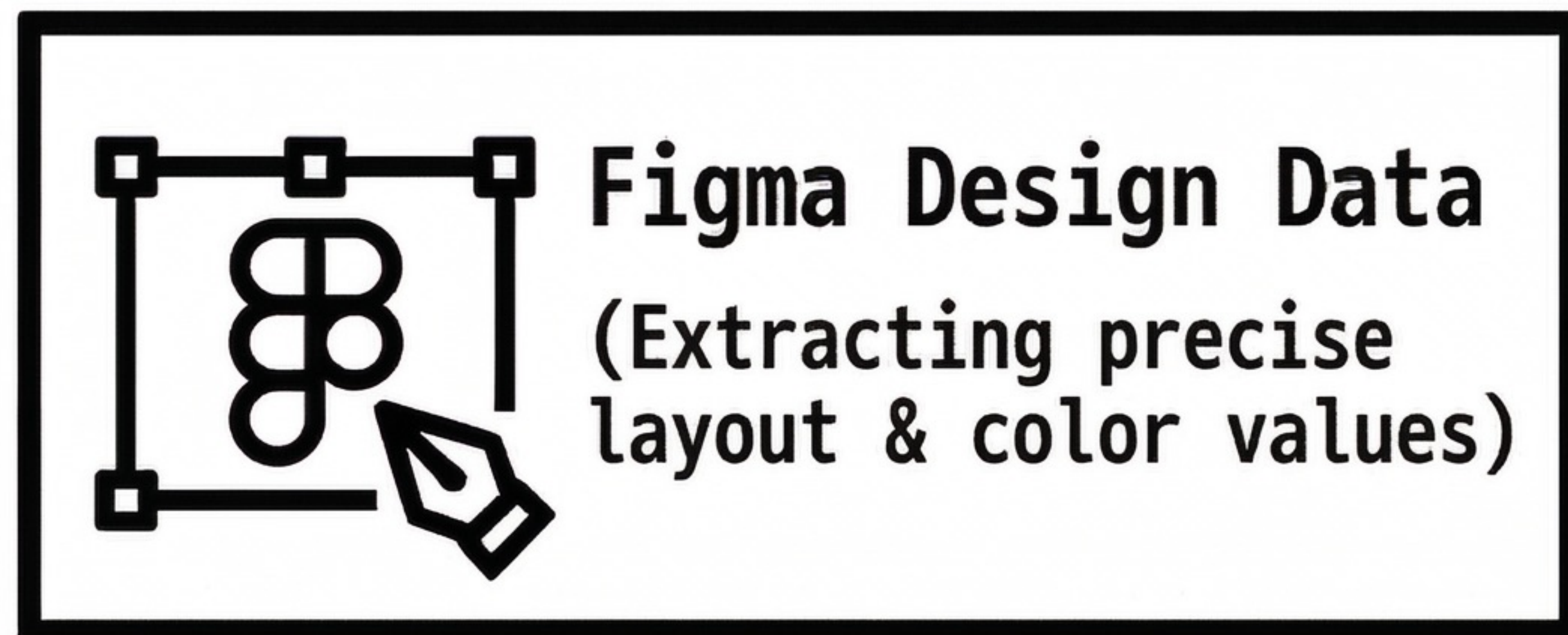
```
# [TARGET] Mediatek Copilot (ask-eda) 全站暗色模式

# Workflow Definition (SOP)
function execute_dark_mode_sync() {
    IF [Need Spec]    -> CALL Figma_MCP
    IF [Need Status] -> CALL Chrome_Screenshot

    # The Core Loop
    COMPARE (Design, Runtime)
    EXECUTE (Code_Refactor)
    VALIDATE (Screenshot == Design)
}
```

透過 Skill 定義 SOP，確保 Agent 確切知曉何時該讀取設計，何時該截圖比對。

STEP 1：規範解析與精準取值

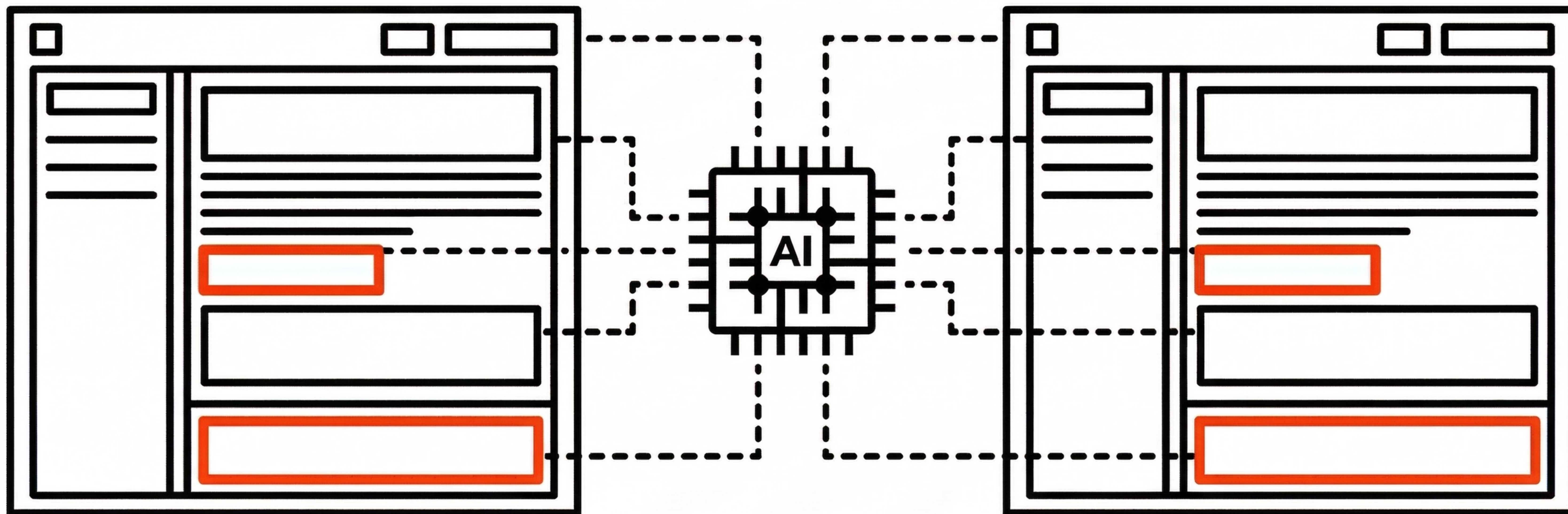


- > **INITIALIZING_CONTEXT...**
- > **讀取專案架構與變體規則... OK**
- > **綁定精準顏色數值... OK**

STEP 2: 像素級視覺比對 (Visual Diffing)

Localhost_Render.png

Figma_Source.design



- [1] 捕捉現況：Agent 截取當前頁面
- [2] 視覺比對：AI 執行像素級對照
- [3] 差異分類：自動歸納背景、文字、邊框之 Visual Gap

STEP 3: 實施變更與自動化重構

[-]



[-]



[+]



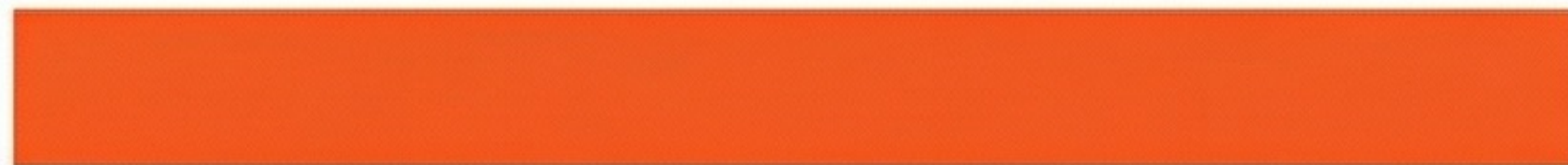
[+]



[-]

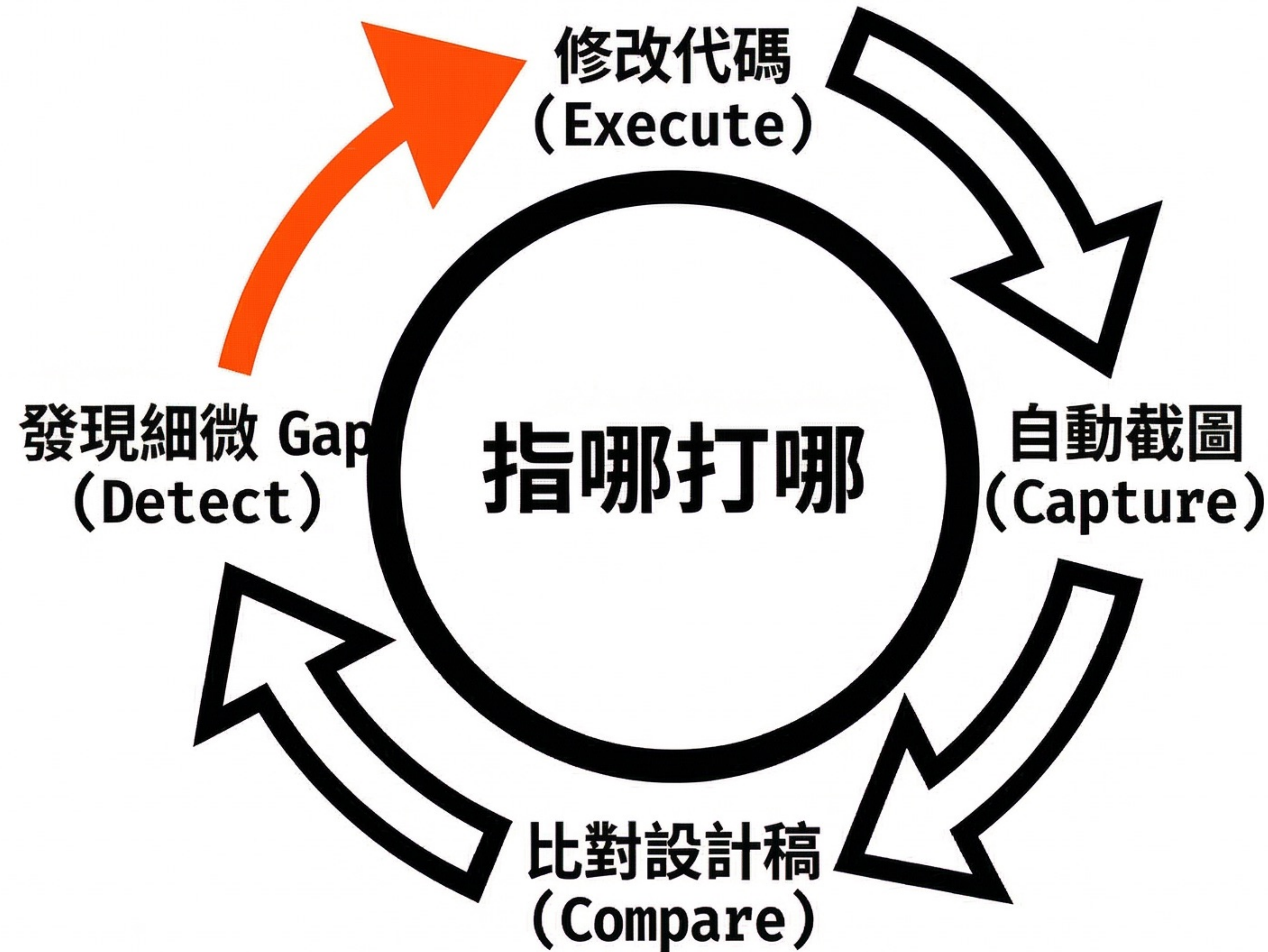


[+]



- > 精確定位相關 Component
- + 添加 Tailwind dark: 類別
- + 更新 SCSS 變數定義
- + 抽離 Hardcoded 數值，建立共用 Tokens
- > SYSTEM_MSG: 寫死頑疾已清除，寫入完成。

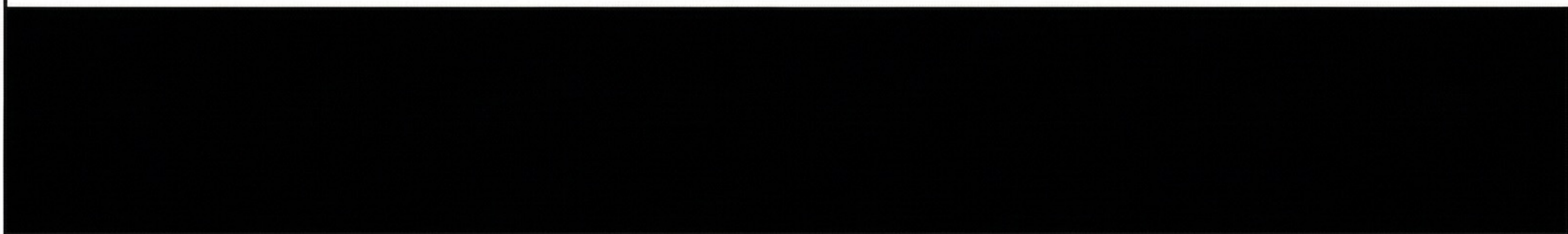
STEP 4: 閉環驗證 (Closed-Loop Validation)



> 循環微調深處數值... 驗證通過 100%。迴圈終止。

系統效能報告：數據成果與擴充潛力

[傳統人工] 1 週 (7 Days)



[Agent 閉環] 1 天 (24 Hours)



[ANALYSIS] 高強度 Token 消耗換取極大的人力時間價值 (ROI 極高)。